

Github repository: <https://github.com/gveettil/movie-accuracy>

1. The goals for your project including what APIs/websites you planned to work with and what data you planned to gather

Our original goals were to webscrape the list of movies based on true stories from the website IMDB to then get the data of those movies from the TMDB api. We were going to calculate a truth scale value using the two Wikipedia summaries from the Wikimedia REST API and the associations of keywords in each summary (which proved to be more difficult than anticipated). We would then compare these scores to assess how similar the movies were to real life and visualize the associations with graphs based on genre, year released and other parameters.

2. The goals that were achieved including what APIs/websites you actually worked with and what data you did gather:

In the end, we analyzed movies based on true stories that we gathered from IMDB using BeautifulSoup. We then got their plot summaries from Wikimedia REST API, and then got the genre, release date, and revenue for each film from the TMDB API. So, we ended up using all of the same APIs/websites as in the original plan, but we did not calculate a “truth score” as originally intended. Instead, we looked at the correlation between the words in each summary and how they matched with genre keywords to match them into categories. We then analyzed how the count, revenue and release date related to these categories, as seen in our calculations/visualizations.

3. The problems that you faced:

The main problem that we faced was accessing the true stories for each movie in order to calculate a “truth score”. For one, there may not be an objective true story for all of the movies, and it may not be well documented on Wikipedia even if there is. For example, some true stories (if they even exist) would be under the names of people whereas some would be under events or dates. So, we instead opted to utilize other data since computing this truth score would be very complex and probably wouldn’t yield very accurate results. Another problem we faced was with our webscraping from IMDB. IMDB uses “lazy loading”, so it only loads what’s on the screen, which posed issues for gathering data from it using only BeautifulSoup, as it didn’t gather all of the titles. So, we had to use Selenium to assist with scrolling through them all.

4. The calculations from the data in the database:

Below is all of our calculations. They were output in a text file so the raw text is included instead of a screenshot:

=====

TRUE STORY MOVIE SUBJECT ANALYSIS - CALCULATIONS

TOTAL MOVIES ANALYZED: 243

1. COUNT OF MOVIES BY SUBJECT CATEGORY

Criminals: 93 movies

Military: 59 movies

Athletes: 33 movies

Musicians: 27 movies

Scientists: 12 movies

Artists & Writers: 6 movies

Other: 5 movies

Politicians: 3 movies

Entertainers: 3 movies

Businesspeople: 2 movies

2. SUBJECT CATEGORIES WITH MOST COMMON MOVIE GENRES

Note: Movies with multiple genres are counted in each genre bucket separately.

Artists & Writers:

Drama: 5 movies

History: 3 movies

Adventure: 2 movies

Action: 1 movies

Music: 1 movies

Romance: 1 movies

Thriller: 1 movies

Athletes:

Drama: 31 movies

History: 11 movies

Action: 6 movies

Adventure: 3 movies

Crime: 3 movies

Comedy: 2 movies

War: 2 movies
Documentary: 1 movies
Family: 1 movies
Music: 1 movies
Romance: 1 movies

Businesspeople:
Drama: 2 movies
Adventure: 1 movies
Comedy: 1 movies
Fantasy: 1 movies
Romance: 1 movies

Criminals:
Drama: 77 movies
History: 32 movies
Action: 29 movies
War: 27 movies
Crime: 23 movies
Thriller: 23 movies
Adventure: 12 movies
Romance: 9 movies
Comedy: 3 movies
Family: 3 movies
TV Movie: 3 movies
Western: 3 movies
Mystery: 2 movies
Animation: 1 movies
Fantasy: 1 movies
Horror: 1 movies
Music: 1 movies

Entertainers:
Drama: 3 movies
Family: 1 movies

Military:
Drama: 54 movies
History: 32 movies
Romance: 13 movies

War: 12 movies
Action: 9 movies
Adventure: 5 movies
Comedy: 4 movies
Family: 4 movies
Thriller: 4 movies
Animation: 1 movies
Crime: 1 movies
Documentary: 1 movies
Fantasy: 1 movies
TV Movie: 1 movies
Western: 1 movies

Musicians:

Drama: 25 movies
Music: 6 movies
Comedy: 5 movies
History: 5 movies
Romance: 4 movies
Thriller: 4 movies
Adventure: 3 movies
War: 3 movies
Action: 2 movies
Crime: 1 movies
Documentary: 1 movies
Horror: 1 movies
Western: 1 movies

Other:

Drama: 5 movies
History: 2 movies
Comedy: 1 movies
Crime: 1 movies
Romance: 1 movies

Politicians:

Drama: 3 movies
Romance: 2 movies
History: 1 movies

Scientists:

Drama: 10 movies

Thriller: 3 movies

Adventure: 2 movies

History: 2 movies

Comedy: 1 movies

Fantasy: 1 movies

Romance: 1 movies

War: 1 movies

3. AVERAGE REVENUE BY SUBJECT CATEGORY (in millions USD)

Other: \$142.36M (based on 3 movies)

Musicians: \$127.51M (based on 22 movies)

Military: \$114.27M (based on 49 movies)

Criminals: \$96.09M (based on 72 movies)

Athletes: \$81.94M (based on 29 movies)

Artists & Writers: \$80.57M (based on 5 movies)

Businesspeople: \$79.03M (based on 2 movies)

Scientists: \$78.70M (based on 8 movies)

Entertainers: \$43.90M (based on 3 movies)

Politicians: \$21.64M (based on 3 movies)

4. REVENUE BY RELEASE YEAR

Data for scatterplot showing box office performance over time.

1985:

Movies: 1

Average Revenue: \$1.30M

Max Revenue: \$1.30M

1993:

Movies: 1

Average Revenue: \$7.00M

Max Revenue: \$7.00M

1995:

Movies: 1

Average Revenue: \$11.50M
Max Revenue: \$11.50M

2000:

Movies: 7
Average Revenue: \$97.49M
Max Revenue: \$256.27M

2001:

Movies: 6
Average Revenue: \$199.10M
Max Revenue: \$449.22M

2002:

Movies: 8
Average Revenue: \$112.60M
Max Revenue: \$352.11M

2003:

Movies: 3
Average Revenue: \$71.76M
Max Revenue: \$148.30M

2004:

Movies: 10
Average Revenue: \$178.91M
Max Revenue: \$610.06M

2005:

Movies: 11
Average Revenue: \$79.64M
Max Revenue: \$218.37M

2006:

Movies: 11
Average Revenue: \$74.40M
Max Revenue: \$307.08M

2007:

Movies: 14

Average Revenue: \$108.78M
Max Revenue: \$456.08M

2008:

Movies: 12
Average Revenue: \$73.01M
Max Revenue: \$255.70M

2009:

Movies: 10
Average Revenue: \$83.09M
Max Revenue: \$309.23M

2010:

Movies: 9
Average Revenue: \$115.59M
Max Revenue: \$414.21M

2011:

Movies: 12
Average Revenue: \$68.49M
Max Revenue: \$216.60M

2012:

Movies: 11
Average Revenue: \$108.05M
Max Revenue: \$275.30M

2013:

Movies: 12
Average Revenue: \$126.38M
Max Revenue: \$407.04M

2014:

Movies: 7
Average Revenue: \$88.42M
Max Revenue: \$233.56M

2015:

Movies: 11

Average Revenue: \$120.22M
Max Revenue: \$532.95M

2016:

Movies: 25
Average Revenue: \$64.33M
Max Revenue: \$311.00M

2017:

Movies: 17
Average Revenue: \$102.31M
Max Revenue: \$527.00M

OVERALL STATISTICS:

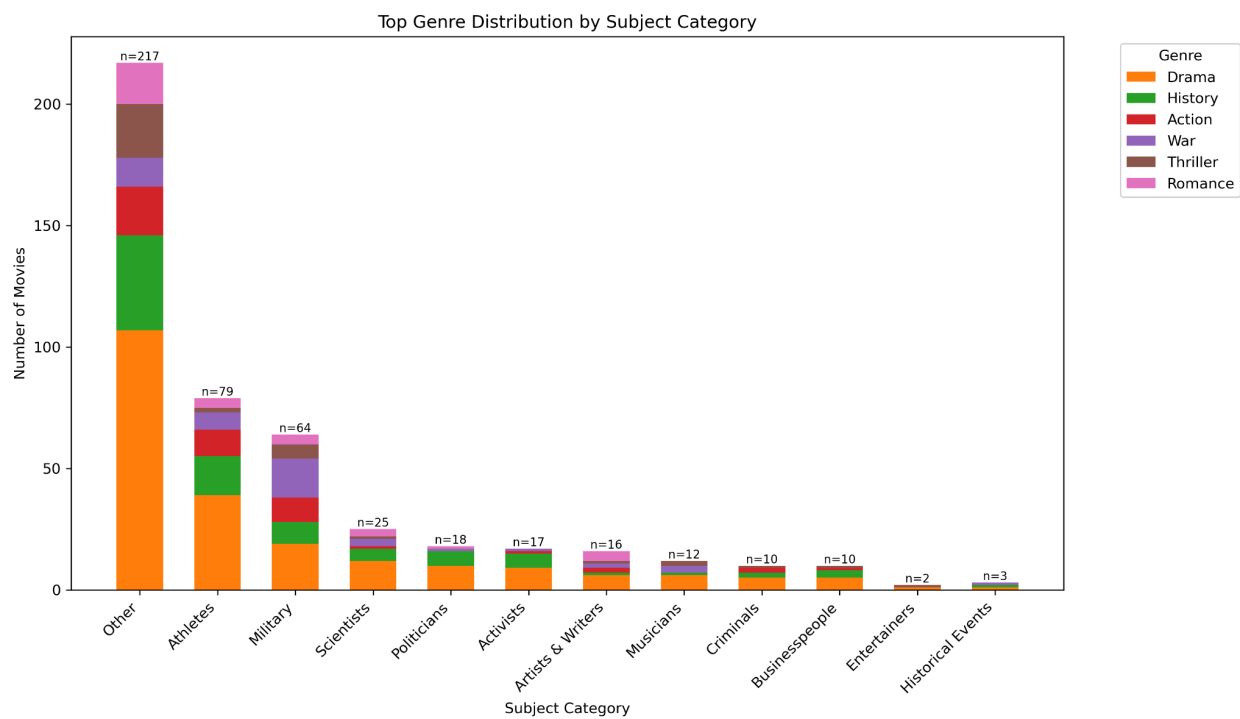
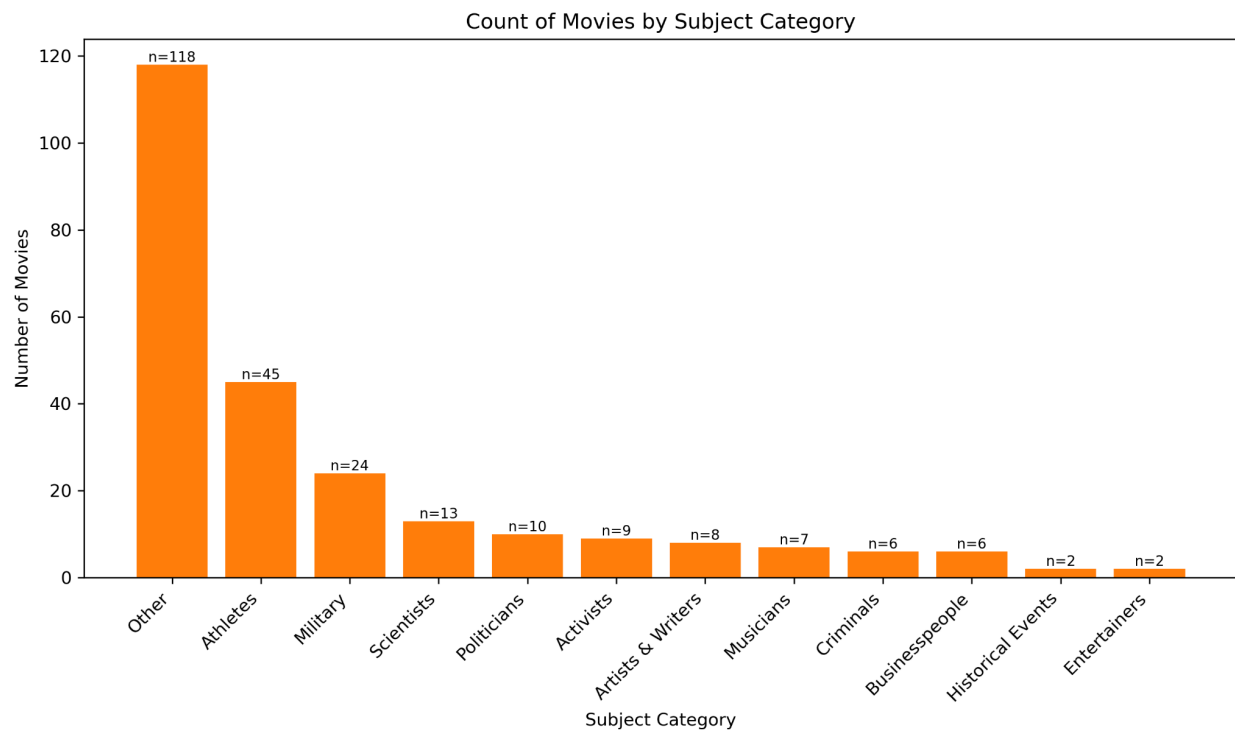
Total movies with revenue data: 199
Year range: 1985 - 2017
Average revenue across all years: \$98.40M
Highest revenue: \$610.06M

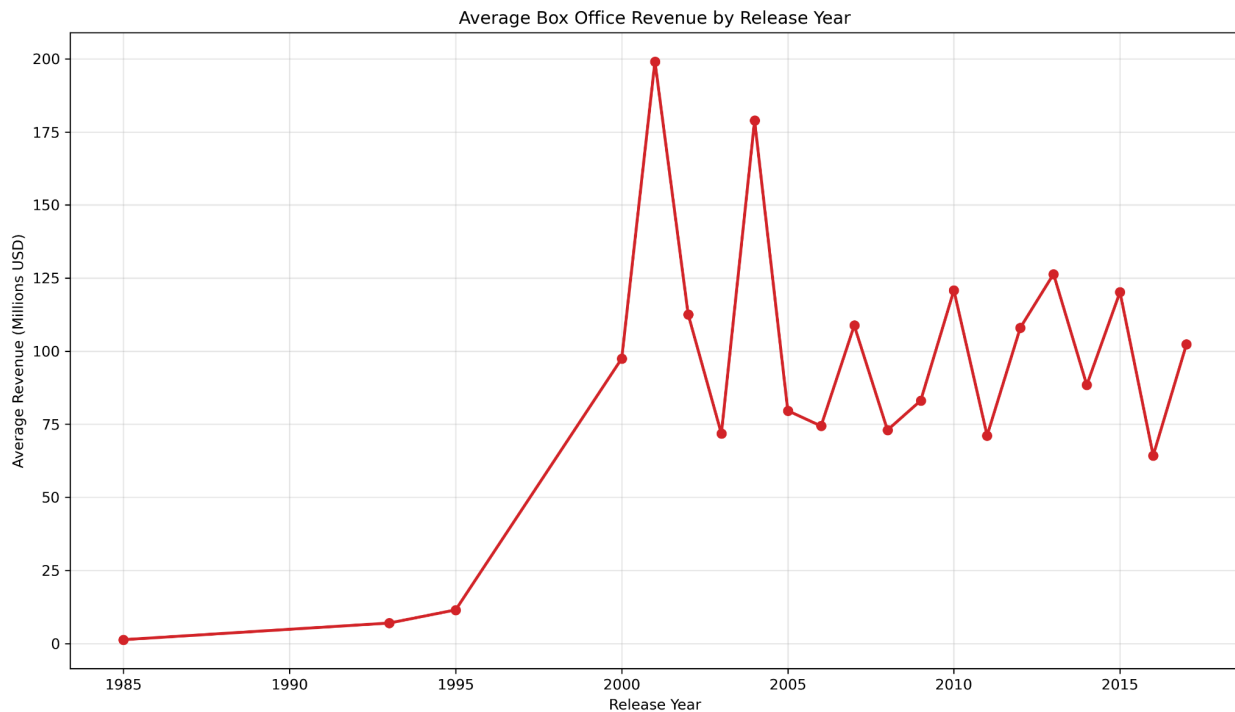
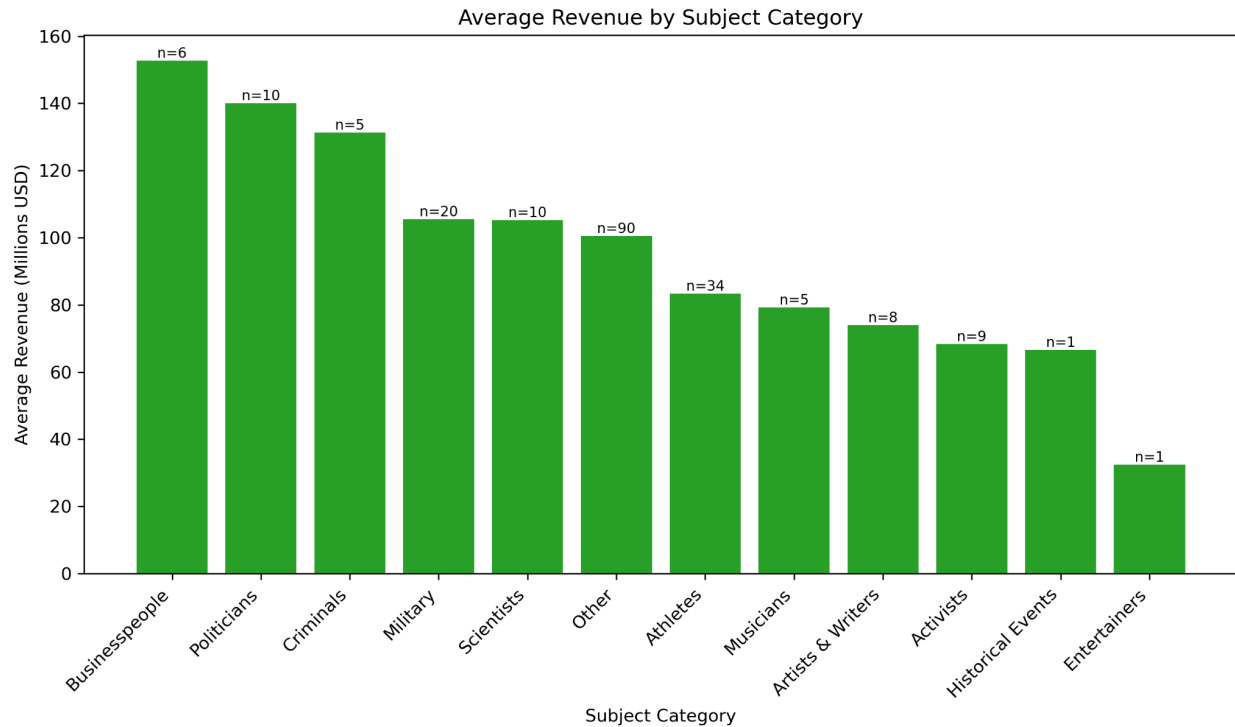
=====

END OF CALCULATIONS

=====

5. The visualization that you created:





6. Instructions for running your code (10 points)

Follow this order to run the project from scratch:

Step 1: Import movie list (run ONCE)

```
python importmovielist.py
```

Step 2: Import plots (run 10+ times until complete)

```
python importplots.py
```

```
python importplots.py
```

Repeat until all movies have plots

Step 3: Import TMDb data (run 10+ times until complete)

```
python tmdbdata.py
```

```
python tmdbdata.py
```

Repeat until all movies are processed

Step 4: Categorize and calculate (run 10+ times until complete)

```
python moviecalc.py
```

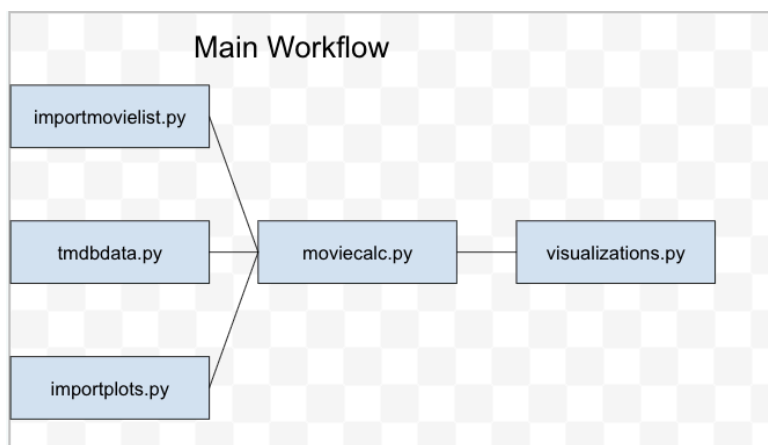
```
python moviecalc.py
```

Repeat until all movies are categorized

Step 5: Generate visualizations (run ONCE)

```
python visualizations.py
```

7. An updated function diagram with the names of each function, the input, and output and who was responsible for that function (20 points)



Gautham: importmovielist.py

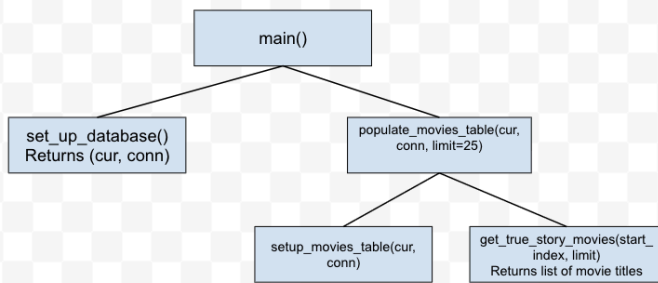
Sam: tmdbdata.py

Colin: importplots.py

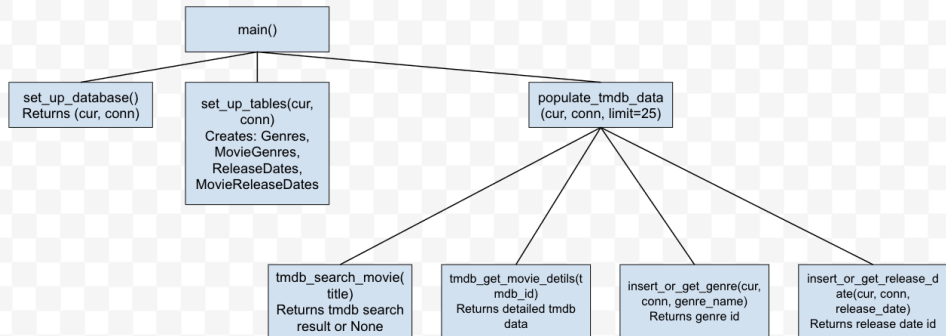
Joint efforts: moviecalc.py and visualizations.py

Function splits for those files are listed below

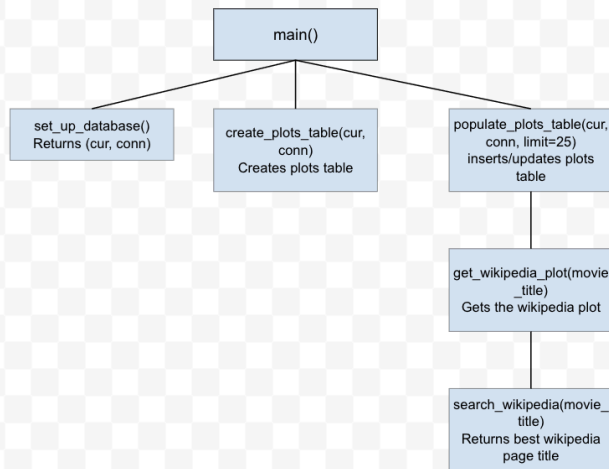
importmovielist.py workflow

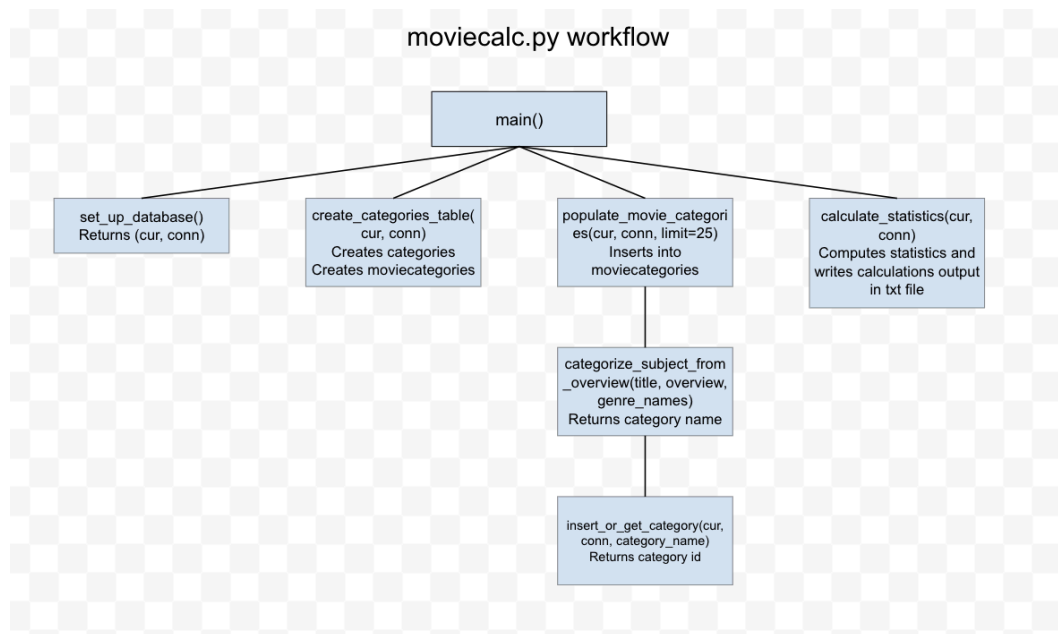


tmdbdata.py workflow



importplots.py workflow



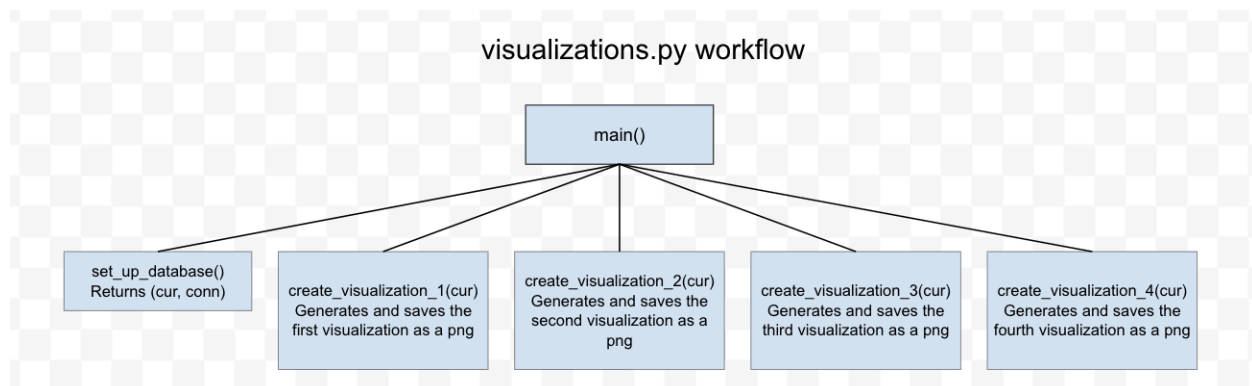


Function Split:

Gautham: set_up_database() and create_categories_table

Sam: categorize_subject_from_overview() and insert_or_get_category()

Colin: populate_movie_categories() and calculate_statistics()



Function Split:

Gautham: set_up_database() and create_visualization_1()

Sam: create_visualization_2() and split create_visualization_4() with Colin

Colin: create_visualization_3() and split create_visualization_4() with Sam

8. You must also clearly document all resources you used. The documentation should be of the following form (20 points)

Date	Issue Description	Location of Resource	Result
12/6/2025	BeautfiulSoup didn't parse all titles from IMDB	Chat GPT	Suggested we use Selenium to scroll, worked successfully
12/5/2025	Having trouble using SQL table syntax	Class Slides	Found the necessary commands in the class slides
12/10/2025	Didn't know how to create the different types of plots correctly in MatPlotlib	Matplotlib.org user guide	Gave us examples of how to make each plot, our plots turned out well
12/8/2025	Needed another overview of how API keys work and how to keep them secure	StackOverflow	Provided a better understanding of how we could use our API keys
12/9/2025	Were a little unsure on how to load in JSON data properly	Discussion 11	Got a good framework/example from discussion that we could adapt